Testing Symbolic Node-Type Embeddings for Algebra Generation with GPT-2

Sawyer Allen

May 8, 2026

Abstract

This project tests whether adding explicit structure to a causal language model improves performance on a synthetic algebra task. The experiment compares a standard GPT-2 small baseline against a structured GPT-2 small model that receives learned node-type embeddings for algebraic prompt tokens. Both models are trained on the same solve-only dataset and evaluated on held-out solve-for-x problems using exact-match accuracy. The final results show no overall improvement from the structured representation: both models achieve the same aggregate exact-match accuracy on the test set. The structured model performs slightly better on easy examples, but slightly worse on hard examples. These results suggest that this simple node-type embedding method is not enough, by itself, to improve exact-match performance in this controlled setting.

1 Introduction

Large language models are usually trained on plain text, even when the task has an underlying structure. Algebra problems are one example where the surface string does not fully represent the mathematical relationships in the expression. For example, the prompt

```
solve 2*x + 3 = 7 for x =>
```

contains variables, constants, operators, equality, and task-specific syntax. A standard language model sees these as text tokens, but does not receive an explicit indication that `x` is a variable, `2` is a constant, or `=` represents an equality relation.

The goal of this project is to test whether adding this kind of coarse symbolic information improves algebra-solving performance. The experiment compares ordinary text-only fine-tuning against a structured version of the same model. The structured model receives the same text as the baseline, but each prompt token is also assigned a learned node-type embedding.

2 Related Work

Recent work on mathematical reasoning in language models has improved performance mainly through math-specific pretraining, post-training, and benchmark construction rather than through small architectural changes. Llemma, DeepSeekMath, and Qwen2.5-Math show that continued training on mathematical text, code, and problem-solution data can improve performance on math benchmarks [1, 2, 3]. Omni-MATH also argues that older math benchmarks are becoming less useful for evaluating stronger models, since many newer systems now perform well on datasets such as GSM8K and MATH [4]. These papers study much larger models and broader tasks than

this project, but they motivate the same basic issue: mathematical text contains structure that ordinary language-model training may not represent reliably.

Recent tool-augmented and neuro-symbolic systems show a stronger version of the same motivation: mathematical reasoning often benefits when language-model outputs are connected to explicit computation or symbolic deduction. ToRA integrates natural-language reasoning with external tools such as computation libraries and symbolic solvers, while MathSensei studies the use of retrieval, Python execution, and Wolfram-Alpha for mathematical reasoning [5, 6]. AlphaGeometry and AlphaGeometry2 use a more specialized neuro-symbolic setup for Olympiad geometry, combining learned components with symbolic deduction and search [7, 8]. These systems are not directly comparable to this experiment because they use tools, symbolic engines, or domain-specific search. The structured GPT-2 model tested here is intentionally simpler: it receives the same prompt text as the baseline and only adds learned coarse node-type embeddings for prompt tokens.

Recent work has also explored symbolic representations inside the model's reasoning format, rather than relying only on external solvers. CoMAT separates mathematical reasoning into symbolic conversion and reasoning execution, using mathematically annotated reasoning steps to reduce ambiguity in the model's intermediate reasoning [9]. This is closer in motivation to the present experiment, although the implementation is still different. CoMAT changes the reasoning format produced by the model, while this project keeps the visible prompt-output text fixed and changes only the input representation through an added embedding channel. The comparison is therefore a narrow test of whether lightweight symbolic type information helps a small causal language model on controlled algebra prompts.

Evaluation of generated mathematical answers is another important issue. Exact string match is easy to compute, but it only measures whether the final extracted answer matches the expected output format. This project uses exact-match accuracy because the task is solve-only and the target answers are written in a fixed canonical form. The comparison is therefore intentionally strict: a model receives credit only when its extracted answer string matches the target answer exactly. This keeps the evaluation simple and consistent with the controlled dataset, but it also means the metric does not measure partial algebraic work or intermediate reasoning.

3 Research Question

The main research question is:

> Does adding coarse node-type information for algebraic symbols improve a causal language model beyond ordinary text-only fine-tuning?

The hypothesis was that node-type information might help the model distinguish between algebraic roles that are not always obvious from token identity alone. For example, the model may benefit from knowing which tokens are variables, constants, operators, equality signs, or punctuation.

4 Dataset

The dataset is a controlled synthetic collection of solve-for-x algebra problems. It is not meant to represent general algebra. Instead, it is designed to test one narrow question: whether adding coarse node-type information to the prompt helps GPT-2 small produce correct final answers on a fixed solve-only task. Every example uses the same prompt format,

`solve <equation> for x =>`

and the target output is a canonical answer such as

`x = 2`

For factorable quadratics with two real integer roots, the target output lists the roots in sorted order:

`x = -1 or x = 3`

The examples were generated programmatically. The generator samples small integer coefficients, constants, shifts, and roots, then constructs equations whose solutions are known by construction. SymPy is used to build and simplify expressions during generation. For example, in the easy linear case the generator samples an integer coefficient a, solution s, and constant b, then forms an equation of the form

$$ax + b = as + b,$$

so that the answer is guaranteed to be $x = s$. For parenthesized and distribution examples, the generator substitutes the sampled solution into the expression to compute the matching right-hand side. For quadratic examples, it samples two integer roots and expands an expression of the form

$$c(x - r_1)(x - r_2) = 0.$$

This ensures that each generated prompt has a known canonical target answer. The implementation uses SymPy operations such as symbolic variables, integer expressions, substitution, simplification, expansion, and string conversion when constructing these examples.

Table 1 shows one example from each solve type used in the experiment.

Solve type	Difficulty	Example prompt	Target output
Easy linear	Easy	`solve 2*x + 3 = 7 for x =>`	`x = 2`
Parenthesized linear	Hard	`solve 3*(x - 1) - 5 = -17 for x =>`	`x = -3`
x terms on both sides	Hard	`solve -x - 6 = x - 8 for x =>`	`x = 1`
Collect like terms	Hard	`solve 3*x - 3*x + 5 = 4*x + 21 for x =>`	`x = -4`
Distribution on both sides	Hard	`solve -3*(x - 1) - 1 = 2*(x + 3) - 4 for x =>`	`x = 0`
Quadratic with two roots	Hard	`solve x**2 - 2*x - 3 = 0 for x =>`	`x = -1 or x = 3`

Table 1: Examples of the solve-for-x problem types used in the synthetic dataset.

The model-visible dataset files are JSONL files containing only two fields: `prompt` and `output`. A typical record is

```
{"prompt": "solve 2*x + 3 = 7 for x =>", "output": "x = 2"}
```

Metadata such as `difficulty` and `solve_kind` is stored separately in sidecar files. These metadata files are used only for grouped evaluation and plotting; they are not included in the training examples and are not visible to either model. Both the baseline and structured model therefore train on the same prompt/output text. The difference is only in the structured model's additional node-type tensor, which is derived from the prompt during preprocessing rather than stored as part of the visible JSONL example.

The final dataset was generated with mixed solve difficulty and deduplicated before splitting. Deduplication uses the visible `prompt/output` pair as the identity of an example. The final run produced 20,968 raw generated examples, from which 12,000 unique prompt/output pairs were kept. These were split into 10,000 training examples, 1,000 validation examples, and 1,000 held-out test examples, with zero duplicate prompt/output pairs crossing between splits.

5 Models

5.1 Baseline Model

The baseline is GPT-2 small fine-tuned directly on prompt-output text. The input text is formed by concatenating the prompt and answer. The prompt is used as context, but prompt tokens are masked from the loss. Therefore, the model is trained only to predict the answer tokens.

5.2 Structured Model

The structured model uses the same GPT-2 small architecture, but adds a learned node-type embedding to each token embedding. The node types are coarse symbolic labels:

- `TASK`
- `VARIABLE`
- `CONSTANT`
- `ADD`
- `MUL`
- `POW`
- `EQUALITY`
- `PUNCT`
- `OTHER`

The structured model does not receive different text from the baseline. Instead, the parser assigns symbolic labels to the prompt tokens, aligns those labels to GPT-2 tokenizer pieces, and creates a parallel `node_type_ids` tensor. The model then sums the normal token embedding and the node-type embedding before passing the result into GPT-2.

The answer tokens are labeled as `OTHER`, and the prompt region is still masked from the loss. This keeps the training objective comparable between the baseline and structured models.

6 Training and Evaluation

Both models are trained on the same train/validation/test split. The final run uses GPT-2 small, one training epoch, batch size 32, evaluation batch size 32, and learning rate 10^{-4}. The baseline and structured models use the same visible prompt/output text, so the comparison isolates the effect of adding the structured input channel.

The baseline model is trained as a standard language model. Each training example is formed by concatenating the prompt, the target output, and an end-of-sequence token. The prompt tokens are included as context, but they are masked out of the loss. This means the model is trained only to predict the answer tokens, not to reconstruct the prompt. The structured model uses the same training objective and the same target labels.

The difference is that the structured model receives an additional learned node-type embedding for each token. The parser assigns coarse algebraic labels to prompt tokens, such as `TASK`, `VARIABLE`, `CONSTANT`, `ADD`, `MUL`, `POW`, `EQUALITY`, and `PUNCT`. These labels are aligned to the GPT-2 tokenizer pieces. If a tokenizer piece cleanly corresponds to one symbolic token, it receives that token's node type; if it merges multiple symbolic roles, it is labeled `OTHER`. Answer tokens and padding positions are also assigned `OTHER`. In the structured model, the learned node-type embedding is added to the normal GPT-2 token embedding before the sequence is passed through the causal language model. This keeps the base model architecture mostly unchanged while giving the model an extra input signal about the algebraic role of each prompt token.

Evaluation is done on the held-out test set using exact-match accuracy. A prediction is counted as correct only if the extracted answer exactly matches the canonical target string. For example, if the target is

```
x = 2
```

then the extracted prediction must match `x = 2` exactly to receive credit.

Generation uses greedy decoding with an algebra-specific token constraint. The decoder only allows tokens made from characters that can appear in the expected answer format, such as variables, digits, signs, equality symbols, parentheses, and simple operators. This prevents the model from drifting into unrelated natural language during evaluation. The generation code also stops once a complete solve-style answer appears.

The comparison script also records whether a correct-looking answer appears as a prefix before extra generated text. This check was included because small causal language models can sometimes produce the right answer first, but then continue generating unrelated algebraic material instead of stopping cleanly. For example, suppose the target answer is

```
x = 2
```

and the raw model output is

```
x = 2 + 3 = 5 or x = -1
```

The full raw output should not be treated as a clean answer, because it contains extra generated text after the initial solution. However, the beginning of the output contains a complete answer span, `x = 2`. The evaluation script extracts the first complete solve-style answer span and compares that extracted answer against the target.

In the final run, this prefix-handling step did not materially affect the results. The prefix exact-match accuracy was 0.0000 for both models, and the raw exact-match accuracy was the same as the extracted exact-match accuracy. Therefore, the reported exact-match results are not being driven by prefix recovery; they mostly reflect direct answer generation.

7 Results

The final evaluation was run on 1,000 held-out test examples. Table 2 summarizes the exact-match accuracy for the baseline GPT-2 model and the structured GPT-2 model. Overall, the two models perform identically: both reach 18.3% exact-match accuracy. This is the main result of the experiment. Adding node-type embeddings did not improve aggregate test accuracy.

Metric	Baseline	Structured
Overall exact match	0.1830	0.1830
Easy exact match	0.2826	0.3370
Hard exact match	0.1729	0.1674

Table 2: Held-out exact-match accuracy on the `solve_mixed_10000_dedup` test set.

Figure 1 shows the same overall result visually. The baseline and structured models are tied at 18.3% accuracy, so there is no evidence of an overall improvement from the structured representation. This matters because the structured model receives additional information about the role of each prompt token, but that extra input channel does not translate into higher aggregate accuracy.

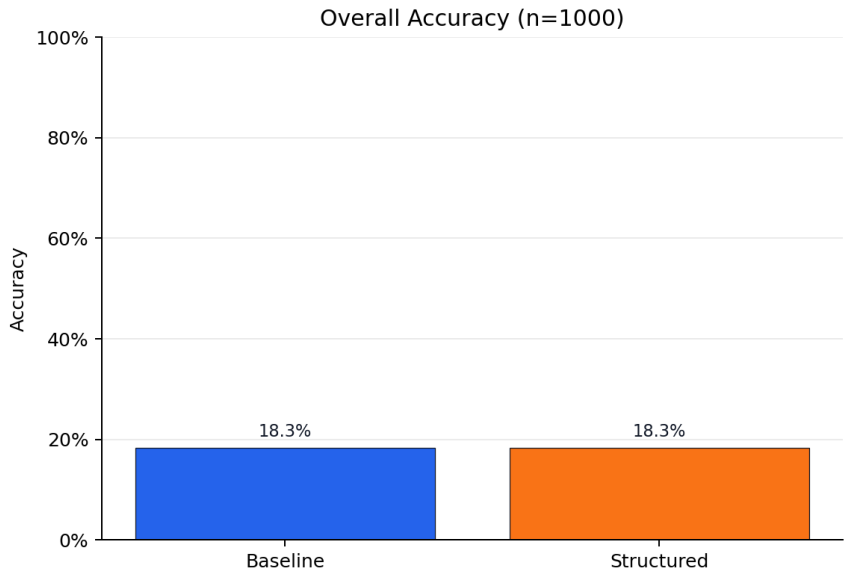

Figure 1: Overall exact-match accuracy on the 1,000-example held-out test set.

Figure 2 breaks the result down by difficulty. The structured model performs better on the easy examples, increasing accuracy from 28.3% to 33.7%. However, on the hard examples, the structured model is slightly worse, decreasing from 17.3% to 16.7%. Since most of the test set is hard examples, with 908 hard examples compared to only 92 easy examples, the easy-example gain is not enough to improve the overall result.

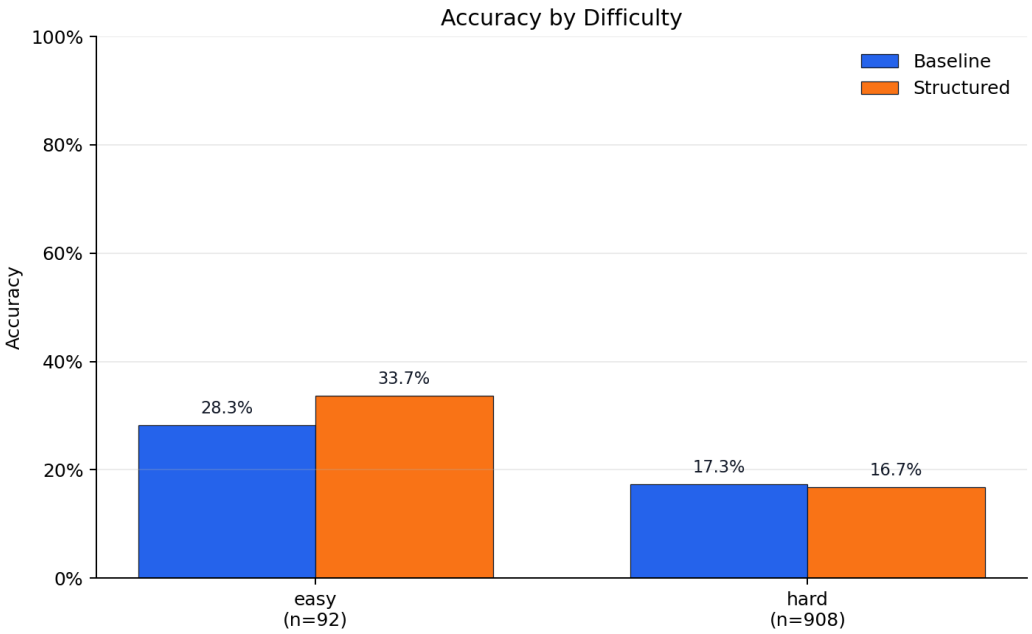

Figure 2: Exact-match accuracy by difficulty level. The structured model improves on easy examples but is slightly worse on hard examples.

Figure 3 gives a more detailed breakdown by equation type. The structured model improves on `linear_easy` and `x_both_sides` examples, ties the baseline on `parenthesized_linear`, and performs worse on `collect_like_terms` and `distribution_both_sides`. Both models score 0.0% on `quadratic_two_real_roots`, but this category only contains 16 test examples, so it should not be interpreted too strongly. The solve-type results suggest that the node-type embeddings changed model behavior in some categories, but not in a consistently beneficial direction.

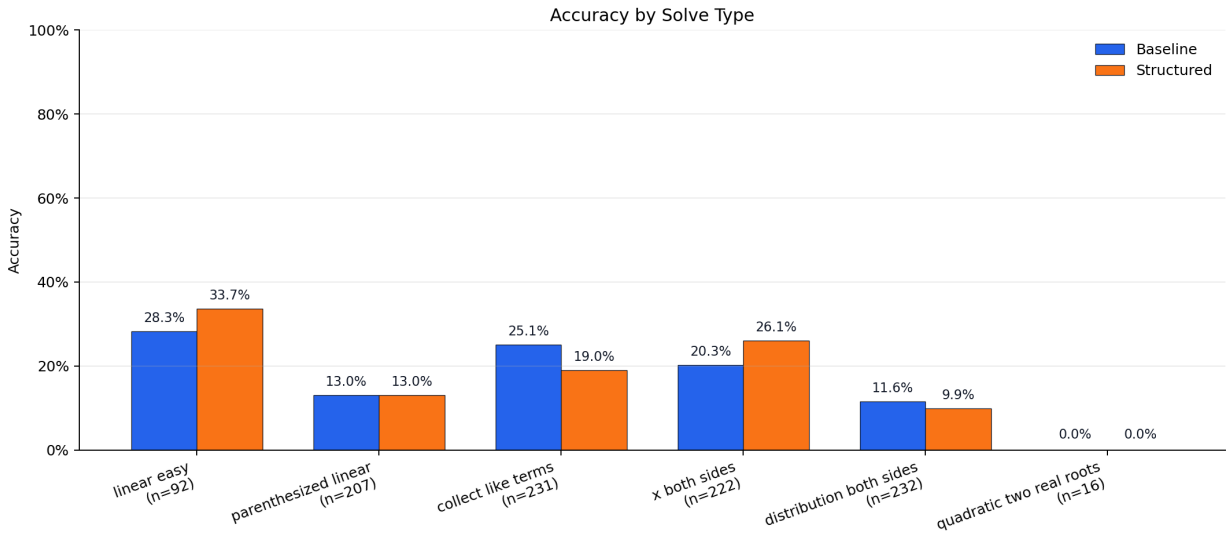

Figure 3: Exact-match accuracy by solve type. The structured model shows mixed category-level behavior rather than a consistent improvement.

Figure 4 shows the masked language modeling loss over training batches. The structured model has a large initial loss spike, but after the first few batches both models settle into a similar loss range. Neither model shows a clear sustained optimization advantage over the other. This is consistent with the final accuracy results: the structured model was trainable and its node-type embedding table was used, but the additional input channel did not produce a clear training or test-performance advantage.

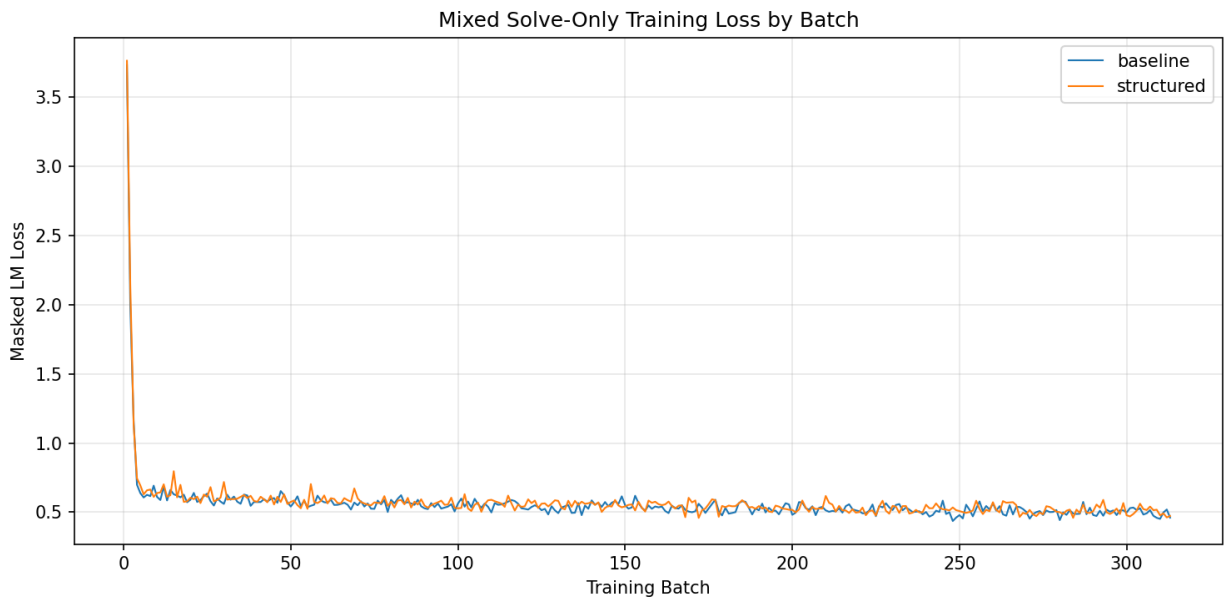

Figure 4: Masked language modeling loss by training batch for the baseline and structured models. After the initial batches, both models train in a similar loss range.

Taken together, the results give mixed evidence for the structured representation. The structured model performs better on some subgroups, especially easy linear equations and equations with x terms on both sides, but it performs worse on other categories and ties the baseline overall. The main conclusion is therefore that adding coarse node-type embeddings did not improve aggregate exact-match accuracy in this experiment, although it did change performance on some equation types.

8 Discussion

The results do not support the hypothesis that coarse node-type embeddings improve exact-match performance on this synthetic solve-for-x task. The structured model was given additional information about the role of each prompt token, but that information did not translate into higher aggregate held-out accuracy.

There are several possible explanations. First, the node types may be too coarse. Labels such as `VARIABLE`, `CONSTANT`, and `ADD` provide basic syntax, but they do not encode deeper algebraic structure such as expression trees, operator precedence, valid transformations, or which terms must be moved across the equality sign.

Second, the model may not have enough training signal to use the added embeddings effectively. The structured embeddings are learned during fine-tuning, but the experiment only uses one epoch. It is possible that longer training or a different learning-rate schedule would produce different results.

Third, the hard examples require multi-step transformations. These include distribution, collecting like terms, moving variables across both sides, and solving quadratics. Coarse token labels may help the model identify parts of the expression, but they do not directly teach the sequence of operations needed to isolate x.

The easy-example result is still worth noting. The structured model performs better on easy exact-match accuracy, which suggests that the node-type information may provide some useful signal for simpler equations. However, this gain does not carry over to harder examples, so it does not improve aggregate test accuracy.

9 Limitations

This experiment has several limitations. First, the model used here is GPT-2 small, so the result should not be treated as evidence about larger or more capable language models. The experiment tests whether a lightweight node-type embedding helps a small language model under one controlled fine-tuning setup. It does not show whether the same idea would help a larger pretrained model, an instruction-tuned model, or a reasoning model that generates longer intermediate reasoning traces before answering.

The dataset is also intentionally narrow. It only contains synthetic solve-for-x equations generated from a fixed grammar. The examples cover easy linear equations, equations with parentheses, equations with variables on both sides, collection of like terms, distribution on both sides, and simple factorable quadratics. This is useful for controlled comparison, but it is still a small slice of algebra. The dataset does not include word problems, rational expressions, systems of equations, inequalities, functions, multi-variable algebra, or problems requiring longer written derivations. Because of this, the results should be interpreted as specific to this synthetic solve-only setting, not as a broad statement about algebra reasoning.

The structured representation is also limited. The node-type labels identify coarse token roles such as `VARIABLE`, `CONSTANT`, `ADD`, `MUL`, and `EQUALITY`, but they do not encode a full expression tree. They do not tell the model which terms belong together, which operations have priority, or what algebraic transformation should be applied next. In other words, the structured model receives extra local token information, but not a complete representation of the equation's mathematical structure.

The experiment also uses only one final training configuration. Both models were trained for one epoch with the same batch size, learning rate, and dataset split. It is possible that the structured model would behave differently with longer training, a different learning rate, a different initialization for the node-type embeddings, more data, or a larger model. This experiment does not explore those alternatives.

Another limitation is that exact-match accuracy is a strict final-answer metric. It does not measure whether the model partially understands the equation or performs some intermediate transformations correctly. A model could make a small arithmetic mistake and receive no credit, while another model could guess the correct final answer without using a reliable procedure. Since the evaluation only scores the extracted final answer, it cannot distinguish between those cases.

Overall, these limitations mean that the result should be read narrowly. The experiment shows that this particular node-type embedding method did not improve aggregate exact-match accuracy for GPT-2 small on this synthetic solve-for-x dataset. It does not rule out richer structured representations, larger models, longer training runs, or reasoning-oriented models that explicitly generate intermediate solution steps.

10 Conclusion

This project tested whether adding node-type embeddings to GPT-2 small improves exact-match performance on a synthetic solve-for-x task. The structured model was implemented so that prompt

tokens received learned algebraic type labels while the visible text and training objective remained comparable to the baseline. On the final held-out test set, the structured model did not improve aggregate exact-match accuracy. It matched the baseline overall, improved slightly on easy examples, and performed slightly worse on hard examples.

The result suggests that simple token-level node-type embeddings are not enough to improve performance in this setting. Future work could test richer structured representations, longer training runs, larger models, and methods that encode expression structure more directly.

References

[1] Zhangir Azerbayev, Hailey Schoelkopf, Keiran Paster, Marco Dos Santos, Stephen McAleer, Qiaochu Jiang, Jia Deng, Stella R. Biderman, and Sean Welleck. *Llemma: An Open Language Model for Mathematics*. International Conference on Learning Representations, 2024.

[2] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*. arXiv preprint arXiv:2402.03300, 2024.

[3] An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, Keming Lu, Mingfeng Xue, Runji Lin, Tianyu Liu, Xingzhang Ren, and Zhenru Zhang. *Qwen2.5-Math Technical Report: Toward Mathematical Expert Model via Self-Improvement*. arXiv preprint arXiv:2409.12122, 2024.

[4] Bofei Gao, Feifan Song, Zhe Yang, Zefan Cai, Yibo Miao, Qingxiu Dong, Lei Li, Chenghao Ma, Liang Chen, Runxin Xu, Zhengyang Tang, Benyou Wang, Daoguang Zan, Shanghaoran Quan, Ge Zhang, Lei Sha, Yichang Zhang, Xuancheng Ren, Tianyu Liu, and Baobao Chang. *Omni-MATH: A Universal Olympiad Level Mathematic Benchmark for Large Language Models*. arXiv preprint arXiv:2410.07985, 2024.

[5] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Minlie Huang, Nan Duan, and Weizhu Chen. *ToRA: A Tool-Integrated Reasoning Agent for Mathematical Problem Solving*. International Conference on Learning Representations, 2024.

[6] Debrup Das, Debopriyo Banerjee, Somak Aditya, and Ashish Kulkarni. *MATHSENSEI: A Tool-Augmented Large Language Model for Mathematical Reasoning*. arXiv preprint arXiv:2402.17231, 2024.

[7] Trieu H. Trinh, Yuhuai Wu, Quoc V. Le, He He, and Thang Luong. *Solving Olympiad Geometry without Human Demonstrations*. Nature, 625:476–482, 2024.

[8] Yuri Chervonyi, Trieu H. Trinh, Miroslav Olšák, Xiaomeng Yang, Hoang Nguyen, Marcelo Menegali, Junehyuk Jung, Vikas Verma, Quoc V. Le, and Thang Luong. *Gold-medalist Performance in Solving Olympiad Geometry with AlphaGeometry2*. arXiv preprint arXiv:2502.03544, 2025.

[9] Joshua Ong Jun Leang, Aryo Pradipta Gema, and Shay B. Cohen. *CoMAT: Chain of Mathematically Annotated Thought Improves Mathematical Reasoning*. Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, pages 20245–20274, 2025. doi:10.18653/v1/2025.emnlp-main.1024.